

Linear Stability Theory, From Scratch

A complete, self-contained guide to formulating and numerically solving the compressible boundary-layer stability problem — as implemented in `pyMack`

Personal reference note

June 30, 2026

Abstract

This note explains — assuming *no prior exposure* to stability theory or spectral methods — everything needed to understand and re-implement a compressible Linear Stability Theory (LST) solver. Every symbol is defined, every matrix is opened up (including the full block structure and which terms carry the eigenvalue), the node ordering and boundary-condition row indices are stated explicitly, and a fully worked numerical example with real solver output is given. All equations, coefficients, and parameter defaults are transcribed from the actual code in `pymack/`, cited by file throughout, so the document doubles as a code $\leftrightarrow$ math map.

Contents

Part I — The idea	2
1 What stability theory asks, and the plan	2
Part II — The mean flow	4
2 Governing equations and non-dimensionalisation	4
3 The base flow	6
Part III — The disturbance equations	7
4 Linearisation and the normal-mode ansatz	8
5 The four disturbance equations	9
Part IV — Turning equations into matrices	10
6 Chebyshev collocation, from scratch	10
7 Assembling the operator and its boundary conditions	11
Part V — The eigenvalue problems	14
8 Temporal problem: a generalised eigenvalue problem	14
9 Spatial problem: a quadratic eigenvalue problem	15

10 Which eigenvalue is physical?	17
11 Worked example: one temporal solve, start to finish	18
12 What the modes <i>are</i> : two instability mechanisms	19
Part VI — Products	21
13 Neutral curves	21
14 N -factor and transition	21
15 Scope and limitations	22
16 Summary and replication checklist	23

How to read this. Sections build strictly on one another. Boxes marked “**What problem is solved here**” state, at each stage, exactly which equation, which boundary conditions, and which unknown (the eigenvalue) are involved. *In plain words* notes give the intuition; grey Remark asides hold secondary or fallback detail you can skip on a first read.

Notation. Overbars are mean (base-flow) quantities; hats are complex disturbance amplitudes. The continuous wall-normal amplitude is $\hat{q}(y) = (\hat{u}, \hat{v}, \hat{T}, \hat{p})$; its discrete (stacked, grid-sampled) form is the vector ϕ . A prime on a mean quantity is $D \equiv d/dy$.

Base flow & transport		Disturbance / eigenvalue	
$\bar{U}, \bar{T}, \bar{\rho}=1/\bar{T}$	mean velocity, temp., density	$\hat{q}; \phi$	amplitude (cont.; discrete), Eq. (15)
$\bar{\mu}, \bar{\kappa}$	viscosity, conductivity	$\hat{u}, \hat{v}, \hat{T}, \hat{p}$	velocity/temp./pressure amplitudes
$C=\bar{\mu}/\bar{T}, K, B$	transport groups, §3	α, ω	wavenumber, frequency
Scales & parameters		$c=\omega/\alpha$	complex phase speed; c_r, c_i its parts
Me, Re, Pr, γ	Mach, Reynolds, Prandtl, c_p/c_v	$\omega_i=\alpha c_i$	temporal growth rate, Eq. (29)
$L^*=\sqrt{\nu_e x/U_e}$	Mack length, Eq. (7)	$\sigma=-\text{Im}(\alpha)$	spatial growth rate, Eq. (34)
$R=\sqrt{Re_x}, F$	Mack Reynolds, reduced freq., Eq. (8)	M_{rel}	relative Mach number, §12
$\alpha_L=\alpha L^*, \delta^*, \theta$	scaled wavenumber, thicknesses	$N, n=N+1$	collocation points; grid size

Operator shorthands: D_1, D_2 derivative matrices (Eq. 26); $\nu_R=1/(Re\bar{\rho})$; $\mathcal{C}_\kappa, \mathcal{C}_d, d_R$ conduction/dissipation prefactors; $\mathcal{L}_c$ conduction operator; A, B (temporal) and C_0, C_1, C_2 (spatial) operator blocks.

Part I — The idea

1 What stability theory asks, and the plan

A laminar boundary layer is a smooth, steady flow that eventually becomes turbulent. The first stage of that process is the amplification of tiny disturbances — noise, roughness, free-stream turbulence — always present in a real flow. *Linear* Stability Theory asks the cleanest version of the question:

If we add one infinitesimally small wave to the laminar flow, does that wave **grow** or **decay** as it travels?

We will write that small wave as $q'(x, y, t)$ — shorthand for the collection of all disturbance quantities (velocity, temperature, pressure; defined precisely in Eq. (14)). “Infinitesimal” is the key word: if q' is small enough, products of small quantities are negligible and its governing equations become *linear*.

Why this becomes an eigenvalue problem. After linearising, the coefficients of the equations depend only on the wall-normal coordinate y (Part III). Because they are *constant in x and t* , substituting a single trial wave $q' \propto e^{i(\alpha x - \omega t)}$ turns every x - and t -derivative into multiplication by a constant; the equations then no longer involve x or t at all, only the wave’s wall-normal shape. The requirement that a *non-zero* shape exist is precisely an **eigenvalue condition**, made concrete in §8.

In plain words. An eigenvalue problem is the statement “ $\mathbf{A}\hat{q} = \lambda\hat{q}$ ”: for most λ the only solution is $\hat{q} = 0$ (no wave), but for special values λ — the eigenvalues — a non-zero shape $\hat{q}$ (the eigenfunction) exists. Here the eigenvalue carries the growth rate, so its sign tells us grow or decay.

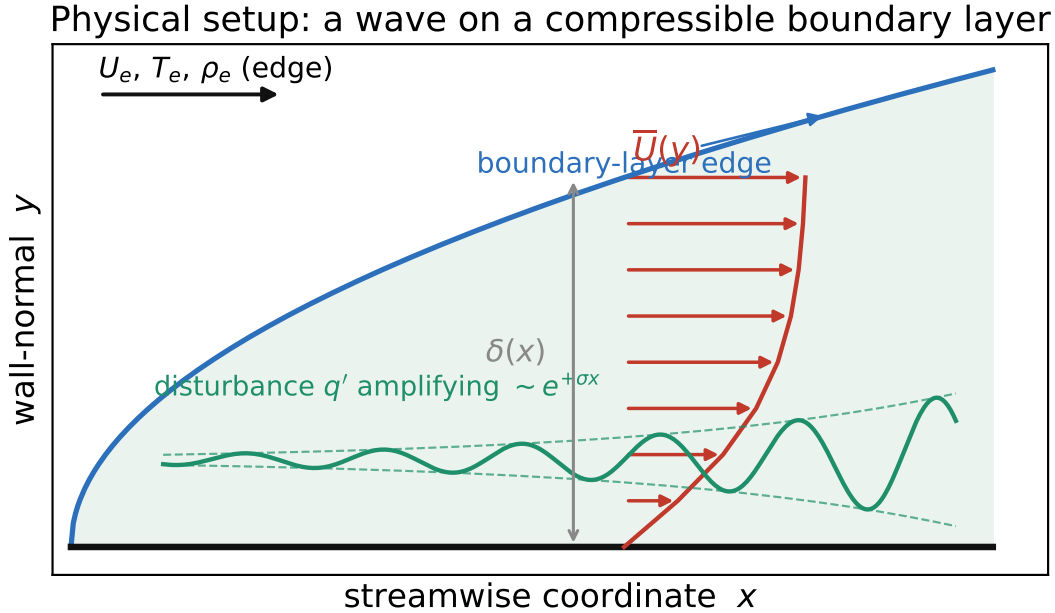


Figure 1: The physical setup. A steady compressible boundary layer grows along a wall; its mean velocity $\bar{U}(y)$ and temperature $\bar{T}(y)$ vary across the layer of thickness $\delta(x)$. LST tracks one small disturbance $q'(x, y, t)$ riding in this layer and asks whether it amplifies downstream.

LST is one stage of a longer story (Figure 2): disturbances first *enter* the layer (receptivity), then grow linearly (LST — this document), then break down nonlinearly to turbulence. We treat only the linear-growth stage; §15 states exactly what that leaves out.

The computation itself is a short pipeline (Figure 3):

1. compute the steady **mean (base) flow** (Part II);
2. **linearise** and insert a wave **ansatz** to get ordinary differential equations in y (Part III);
3. **discretise** those ODEs into matrices with Chebyshev collocation (Part IV);
4. solve the resulting **eigenvalue problem** for the growth rate (Part V);
5. sweep parameters to draw **neutral curves** and integrate **N -factors** (Part VI).

Where LST sits: this document covers the highlighted (linear-growth) stage

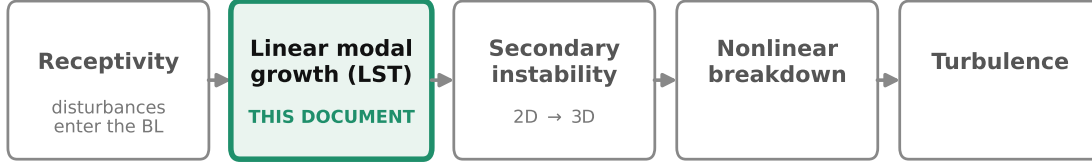


Figure 2: The transition process. This document covers the highlighted linear-modal-growth stage (which produces the e^N amplification); receptivity and nonlinear breakdown are separate problems (§15).

Part II — The mean flow

The disturbance rides on a steady laminar flow, so we need that flow — and its y -derivatives — before anything else. This Part computes it.

2 Governing equations and non-dimensionalisation

The flow obeys the compressible Navier–Stokes equations — conservation of mass, momentum, and energy for a viscous, heat-conducting perfect gas:

$$\frac{D\rho}{Dt} + \rho \nabla \cdot \mathbf{u} = 0, \quad (\text{mass}) \quad (1)$$

$$\rho \frac{D\mathbf{u}}{Dt} = -\nabla p + \nabla \cdot \boldsymbol{\tau}, \quad (\text{momentum}) \quad (2)$$

$$\rho c_p \frac{DT}{Dt} = \frac{Dp}{Dt} + \nabla \cdot (\kappa \nabla T) + \Phi, \quad (\text{energy}) \quad (3)$$

$$p = \rho RT, \quad (\text{state}) \quad (4)$$

where $\mathbf{u} = (u, v)$ is velocity, ρ density, T temperature, p pressure, μ viscosity, κ conductivity, c_p specific heat, R the gas constant, and $D/Dt = \partial_t + \mathbf{u} \cdot \nabla$ the material derivative. The two remaining symbols are the Newtonian **viscous stress tensor** and the **dissipation function**,

$$\boldsymbol{\tau} = \mu \left(\nabla \mathbf{u} + \nabla \mathbf{u}^T - \frac{2}{3} (\nabla \cdot \mathbf{u}) \mathbf{I} \right), \quad \Phi = \boldsymbol{\tau} : \nabla \mathbf{u}. \quad (5)$$

We will not need their closed form again: all that matters downstream is that they are linear in velocity gradients. These four equations are applied *twice* — once to obtain the steady mean flow (this Part), once linearised for the disturbance (Part III).

Non-dimensionalisation and default gas constants. Velocities are scaled by the edge speed U_e , temperature by T_e , density by ρ_e , viscosity by μ_e , pressure by $\rho_e U_e^2$. Because the gas is perfect and pressure is nearly uniform across the layer, $\bar{p} = 1/\bar{T}$ in these units. Three dimensionless groups control everything,

$$M_e = \frac{U_e}{\sqrt{\gamma R T_e}}, \quad Re = \frac{\rho_e U_e L_{\text{ref}}}{\mu_e}, \quad Pr = \frac{\mu_e c_p}{\kappa_e}, \quad (6)$$

End-to-end LST workflow in pyMack

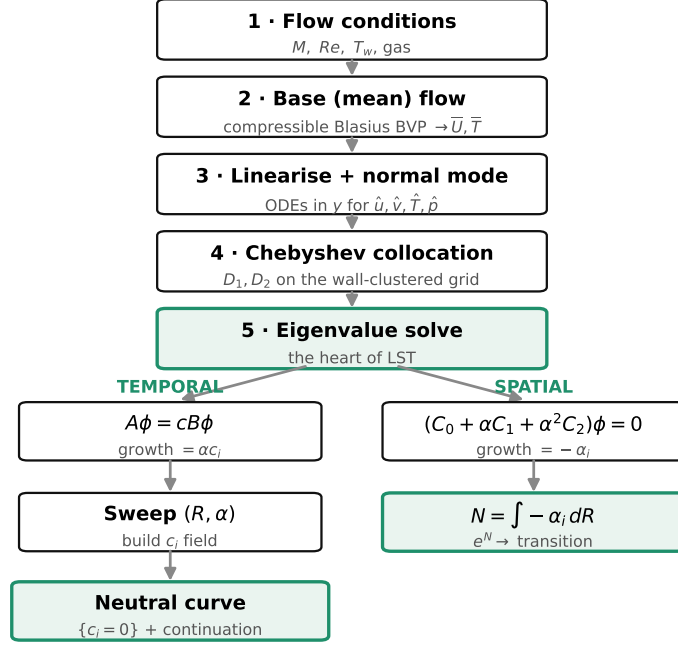


Figure 3: End-to-end workflow. The single eigenvalue solve (stage 5) forks into a *temporal* route (real wavenumber α in, complex phase speed c out; used for neutral curves) and a *spatial* route (real frequency ω in, complex α out; used for N -factors).

with default air values $\gamma = 1.4$, $Pr = 0.72$, Sutherland constant $S = 110.4$ K, and power-law exponent $\omega_\mu \approx 0.7$ when the power-law viscosity is used.

The Mack length and Reynolds scales. Boundary layers grow like $\sqrt{x}$, so the natural local scale at station x is

$$L^* = \sqrt{\frac{\nu_e x}{U_e}}, \quad \nu_e = \frac{\mu_e}{\rho_e}. \quad (7)$$

With $L_{\text{ref}} = L^*$ the Reynolds number becomes the square root of the usual one, and a fixed physical frequency f maps to a reduced frequency F (both *first used in Part VI*, where neutral curves and N -factors consume them):

$$R = \frac{U_e L^*}{\nu_e} = \sqrt{Re_x}, \quad Re_x = \frac{U_e x}{\nu_e}; \quad F = \frac{2\pi f \nu_e}{U_e^2}; \quad \omega = F R, \quad \alpha_L = \alpha L^*. \quad (8)$$

R is the horizontal axis of every neutral curve and α_L the vertical axis. Conversions live in `pymack/scales.py`; the solver itself is dimensionless. A profile may be stored on a y/δ^* or a y/L^* grid; the constant δ^*/L^* converts between them, with $D \mapsto (\delta^*/L^*)^{-1}D$.

3 The base flow

What problem is solved here. Solve a *boundary-value problem* (BVP) — not an eigenvalue problem — for the steady laminar profiles $\bar{U}(y), \bar{T}(y), \bar{\rho}(y), \bar{\mu}(y)$ and their y -derivatives. A BVP fixes the data at both ends ($y = 0$ and $y \rightarrow \infty$) and has a single solution; an eigenvalue problem instead leaves a parameter free and asks for the special values that admit a non-zero solution. These profiles become the known coefficients of the disturbance equations.

For a flat plate the mean flow is *self-similar*: profiles at different x collapse onto one curve in a stretched coordinate η . With the dimensionless stream function $f(\eta)$, $f' = \bar{U} = u/U_e$ (a prime is $d/d\eta$), momentum and energy reduce to two coupled ODEs,

$$(C f'')' + f f'' = 0, \quad (9)$$

$$(B \bar{T}')' + f \bar{T}' + (\gamma - 1) M_e^2 C (f'')^2 = 0, \quad (10)$$

with transport groups

$$C = \frac{\mu/\mu_e}{T/T_e} \text{ (Chapman–Rubesin)}, \quad K = \frac{\kappa}{c_p \mu_e}, \quad B = \frac{K}{T} \left(= \frac{C}{Pr} \text{ at constant } Pr \right). \quad (11)$$

The last term in (10) is aerodynamic heating — it makes a hypersonic layer hot near the wall and couples energy to velocity. The conductivity model is $\kappa = \mu c_p / Pr$ (so $\bar{\kappa} = \bar{\mu} / Pr$ at constant Pr); the transport derivatives $d\mu/dT$ etc. are taken analytically from the chosen viscosity law.

Wall and edge conditions.

$$\underbrace{f(0) = 0, f'(0) = 0}_{\text{no slip}}, \quad \underbrace{f'(\infty) = 1, \bar{T}(\infty) = 1}_{\text{match the edge}}, \quad \begin{cases} \bar{T}'(0) = 0 & \text{(adiabatic wall)} \\ \bar{T}(0) = T_w/T_e & \text{(isothermal wall)} \end{cases} \quad (12)$$

For an adiabatic wall the recovered temperature is the standard result (stated, not derived) $T_{aw}/T_e = 1 + \frac{1}{2}(\gamma - 1)\sqrt{Pr} M_e^2$. (For a cone, the edge state itself comes first, from the Taylor–Maccoll equation behind the conical shock; `taylor_maccoll_edge_state.py`.)

Solution and the coordinate transform. Equations (9)–(10) are solved as a 5-state first-order system $[f, \bar{U}, \bar{U}', \bar{T}, \bar{T}']$ by `scipy.integrate.solve_bvp` (collocation + Newton; `pymack/baseflow.py`), integrated to $\eta_{\max} \approx 40$ (well beyond the edge). The similarity coordinate maps to the physical one by

$$\frac{y}{L^*} = \sqrt{2} \int_0^\eta \frac{\bar{T}}{T_e} d\eta', \quad \frac{\delta^*}{L^*} = \sqrt{2} \int_0^\infty (\bar{T} - \bar{U}) d\eta, \quad \frac{\theta}{L^*} = \sqrt{2} \int_0^\infty \bar{U}(1 - \bar{U}) d\eta, \quad (13)$$

where the displacement thickness δ^* measures how far the wall effectively pushes the outer flow outward and the momentum thickness θ the momentum deficit. The solver returns, on any grid, $\bar{U}, \bar{U}', \bar{U}'', \bar{T}, \bar{T}', \bar{T}'', \bar{\rho}, \bar{\mu}, d\mu/dT, d^2\mu/dT^2, \bar{\kappa}, d\kappa/dT, d^2\kappa/dT^2$.

Two base-flow implementations. `baseflow.py` contains two equivalent self-similar formulations sharing the 5-ODE structure above. Equations (9)–(10) and the $\sqrt{2}$ transform (13) are the `CompressibleBlasiusProfile` form. The temporal solver and the worked example of §11 use `make_flatplate_profile` (`FlatPlateProfile`; Ozgen & Kircali transport laws), an alternative normalisation ($f' = \bar{U}/\bar{T}$, $\frac{1}{2}$ factors, $y/L^* = \eta$, no $\sqrt{2}$). The two give the same physical profiles; only the similarity bookkeeping differs.

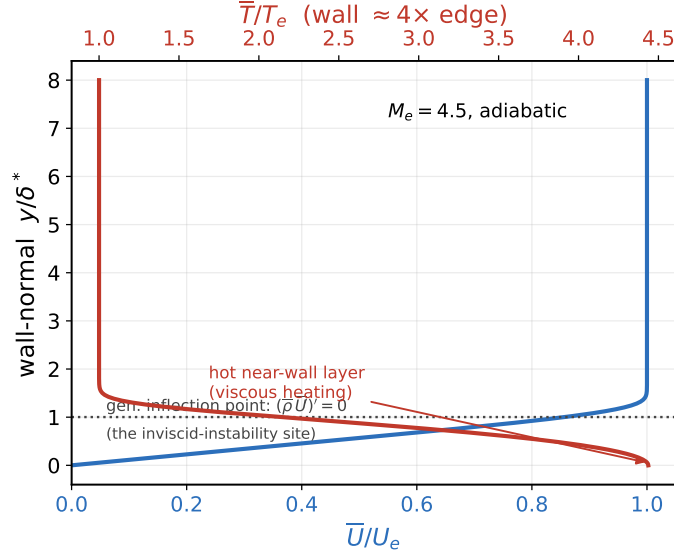


Figure 4: A computed compressible base flow ($M_e = 4.5$, adiabatic). $\bar{U}(y)/U_e$ rises monotonically to the edge; $\bar{T}(y)$ bulges near the wall from viscous heating. These curves and their y -derivatives are the known coefficients $\bar{U}, \bar{T}, \dots$ in the disturbance equations. The dotted line marks the generalised inflection point (§12), the seat of the inviscid instability.

Part III — The disturbance equations

We now perturb the mean flow by an infinitesimal wave and reduce the governing PDEs to ordinary differential equations in y — the equations the rest of the document discretises and solves.

From the governing equations to the matrix problem the code solves. Part III, and the two Parts after it, are a single chain of transformations carrying the *full governing equations* (top — nonlinear, unsolvable by hand) into the *one matrix problem the code actually solves* (bottom — boxed). Those two are the forms worth remembering; everything in between is how we get from the first to the last.

$$\begin{aligned} \partial_t \rho + \nabla \cdot (\rho \mathbf{u}) &= 0, & \rho \frac{D\mathbf{u}}{Dt} &= -\nabla p + \nabla \cdot \boldsymbol{\tau}, \\ \rho c_p \frac{DT}{Dt} &= \frac{Dp}{Dt} + \nabla \cdot (\kappa \nabla T) + \Phi, & p &= \rho RT \end{aligned}$$

the **governing equations**: nonlinear, compressible Navier–Stokes in (x, y, t) for $\mathbf{q} = (u, v, T, p, \rho)$ — Eqs. (1)–(4)

↓ split $\mathbf{q} = \bar{\mathbf{q}} + \mathbf{q}'$, subtract the mean (it solves the equations on its own), drop products of the small primes (§4, Steps 1–2)

linear PDEs for the disturbance $\mathbf{q}' = (u', v', T', p', \rho')$ in (x, y, t)
every coefficient is now a *known* mean profile $\bar{U}(y), \bar{T}(y), \dots$

↓ parallel-flow assumption, then the normal-mode ansatz $\mathbf{q}' = \hat{q}(y) e^{i(\alpha x - \omega t)}$, so $\partial_x \rightarrow i\alpha$, $\partial_t \rightarrow -i\omega$ (§4, Steps 3–5)

$$\mathbf{L}(\alpha, c; y, D) \hat{q}(y) = \mathbf{0}$$

four coupled **ODEs** in y for the wall-normal shape $\hat{q} = (\hat{u}, \hat{v}, \hat{T}, \hat{p})$ — Eqs. (17)–(20)

↓ Chebyshev collocation: $D \rightarrow D_1$, $D^2 \rightarrow D_2$, each mean coefficient $\rightarrow$ a diagonal matrix, the four fields stacked into one vector (§6–7)

$$\boxed{A(\alpha) \phi = c B(\alpha) \phi} \quad A, B \in \mathbb{C}^{4n \times 4n}$$

the **discrete generalised eigenvalue problem** `temporal_solver.py` assembles and solves — Eq. (28)

The top line is a system of nonlinear partial differential equations no one can solve by hand; the bottom line is finite linear algebra. The eigenvalue carries the answer: `temporal_solver.py` fills the two matrices A, B and calls `scipy.linalg.eig(A, B)`, which returns the complex phase speed c (the eigenvalue) and the grid-sampled mode shape ϕ (the eigenvector). Holding α real and solving for c is the *temporal* problem (§8); holding ω real and regrouping the same ODEs by powers of α turns the last line into the *spatial* quadratic problem (§9). The rest of Part III derives the two middle lines; Parts IV–V derive the last.

4 Linearisation and the normal-mode ansatz

Step 1 — split into mean plus disturbance.

$$u = \bar{U}(y) + u', \quad v = v', \quad T = \bar{T}(y) + T', \quad p = \bar{p} + p', \quad \rho = \bar{\rho}(y) + \rho', \quad |\cdot'| \ll |\cdot|. \quad (14)$$

($\bar{v} = 0$ for a parallel flow.) The collection $q' = (u', v', T', p', \rho')$ is the disturbance.

Step 2 — drop products of disturbances. Substitute (14) into (1)–(4). The mean satisfies the equations by itself and cancels; any term with two primes is the product of two small numbers and is dropped. What remains is *linear* in q' with coefficients made of the known mean profiles.

Step 3 — parallel-flow assumption. The layer grows slowly, so locally $\bar{q} = \bar{q}(y)$ only; the coefficients depend on y alone.

Caveat. This neglects the $O(1/R)$ streamwise growth of the mean flow and the eigenfunction; the rigorous remedy is the Parabolised Stability Equations (PSE), which march the disturbance downstream retaining that slow x -variation (§15).

Step 4 — the normal-mode ansatz.

$$q'(x, y, t) = \hat{q}(y) e^{i(\alpha x - \omega t)}, \quad \hat{q} = (\hat{u}, \hat{v}, \hat{T}, \hat{p})^\top, \quad (15)$$

with α the streamwise wavenumber ($2\pi/\alpha = \text{wavelength}$), ω the frequency, $c = \omega/\alpha$ the complex phase speed, $\hat{q}(y)$ the wall-normal shape.

2D wave. We take spanwise wavenumber $\beta = 0$. A general oblique wave $\propto e^{i(\alpha x + \beta z - \omega t)}$ travels at angle $\psi = \arctan(\beta/\alpha)$; compressibility breaks Squire’s theorem, so the *first* mode is most unstable oblique ($\psi \sim 55^\circ$) while the *second* mode peaks at $\psi = 0$. The 2D solver thus captures the second mode faithfully but under-predicts the first (§15).

Substitution gives $\partial_x \mapsto i\alpha$, $\partial_t \mapsto -i\omega$, $\partial_y \mapsto D \equiv d/dy$, collapsing the PDEs to coupled ODEs in y .

Step 5 — eliminate density. The linearised state law gives $\rho'/\bar{\rho} = \gamma M_e^2 \hat{p} - \hat{T}/\bar{T}$, removing ρ' . The state then has four fields $\hat{u}, \hat{v}, \hat{T}, \hat{p}$, with $\hat{p}$ the pressure amplitude scaled by $\rho_e U_e^2$ (so $\gamma M_e^2 \hat{p} = p'/p_e$).

A worked linearisation (continuity), so the mechanics are explicit. Take mass conservation (1), $\partial_t \rho + \nabla \cdot (\rho \mathbf{u}) = 0$. Insert (14); the steady mean satisfies $\partial_x(\bar{\rho}\bar{U}) = 0$ and cancels. Keeping only terms *linear* in primes:

$$\partial_t \rho' + \partial_x(\bar{\rho}\bar{U})' + \partial_x(\rho' \cdot 0) \dots \Rightarrow \partial_t \rho' + \bar{U} \partial_x \rho' + \bar{\rho} \partial_x u' + \partial_y(\bar{\rho} v') = 0. \quad (16)$$

Now apply the ansatz ($\partial_t \rightarrow -i\omega = -i\alpha c$, $\partial_x \rightarrow i\alpha$, $\partial_y \rightarrow D$) and substitute ρ' from Step 5; collecting terms and dividing by $\bar{\rho}$ gives exactly Eq. (17) below. *The other three equations follow by the identical procedure on (2)–(3); the algebra is mechanical but long, so only the result is printed.*

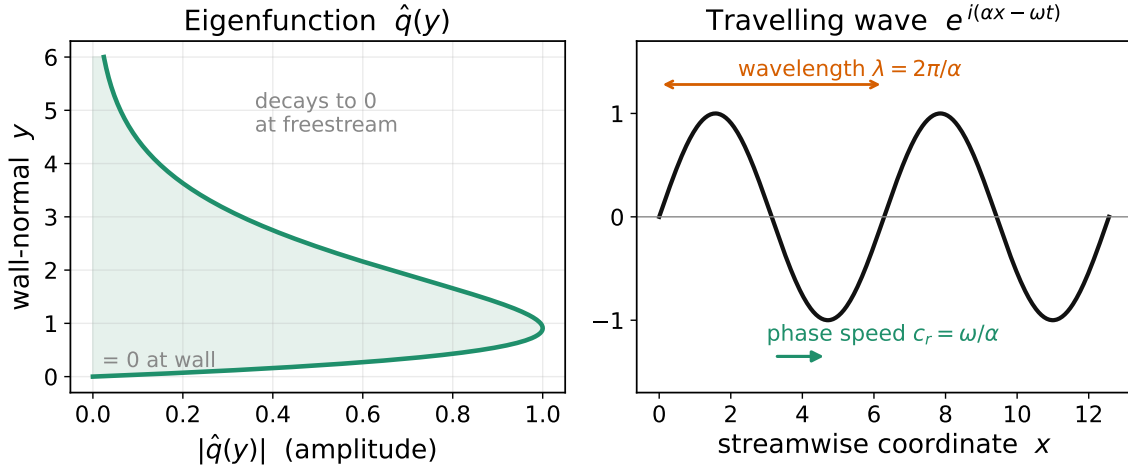


Figure 5: The ansatz (15) factorises a disturbance $q' = \hat{q}(y) e^{i(\alpha x - \omega t)}$ into a wall-normal *shape* $\hat{q}(y)$ (left; zero at the wall, decaying at the edge — the eigenfunction) times a travelling wave in x (right; wavelength $2\pi/\alpha$, phase speed $c_r = \text{Re}(c)$).

5 The four disturbance equations

What problem is solved here. These are the linear ODEs in y for $\hat{u}, \hat{v}, \hat{T}, \hat{p}$. Fixing one of α or ω real makes the other (through c or α) the eigenvalue. Every overbarred quantity is a known base-flow coefficient; all transport derivatives ($d\mu/dT$, $d\kappa/dT$, ...) are evaluated at the local mean temperature $\bar{T}(y)$ and are therefore known functions of y .

The form assembled in `pymack/temporal_solver.py` follows. Momentum and energy are divided by $\bar{\rho}$; write the viscous prefactor $\nu_R \equiv 1/(Re \bar{\rho})$. Throughout, every $\bar{\rho}$ is the base-flow density, i.e. the diagonal field $\bar{\rho}(y) = 1/\bar{T}(y)$ — this single substitution applies uniformly inside ν_R , $\mathcal{C}_\kappa$, and $\mathcal{C}_d$ below.

(C) Continuity + equation of state.

$$i\alpha \hat{u} + D\hat{v} - \frac{\bar{T}'}{\bar{T}} \hat{v} + i\alpha(\bar{U} - c) \left(\gamma M_e^2 \hat{p} - \frac{\hat{T}}{\bar{T}} \right) = 0. \quad (17)$$

(X) **Streamwise momentum.**

$$\begin{aligned} i\alpha(\bar{U} - c)\hat{u} + \bar{U}'\hat{v} + i\alpha\bar{\rho}^{-1}\hat{p} = \nu_R \Big[\bar{\mu}(D^2 - \frac{4}{3}\alpha^2)\hat{u} + \bar{\mu}'D\hat{u} + i\alpha(\frac{1}{3}\bar{\mu}D + \bar{\mu}')\hat{v} \\ + \frac{d\mu}{dT}(\bar{U}'' + \bar{U}'D)\hat{T} + \frac{d^2\mu}{dT^2}\bar{U}'\bar{T}'\hat{T} \Big]. \end{aligned} \quad (18)$$

(Y) **Wall-normal momentum.**

$$i\alpha(\bar{U} - c)\hat{v} + \bar{\rho}^{-1}D\hat{p} = \nu_R \Big[\bar{\mu}(\frac{4}{3}D^2 - \alpha^2)\hat{v} + \frac{4}{3}\bar{\mu}'D\hat{v} + i\alpha(\frac{1}{3}\bar{\mu}D - \frac{2}{3}\bar{\mu}')\hat{u} + i\alpha\frac{d\mu}{dT}\bar{U}'\hat{T} \Big]. \quad (19)$$

(E) **Energy (temperature).**

$$i\alpha(\bar{U} - c)\hat{T} + \bar{T}'\hat{v} + (\gamma - 1)\bar{\rho}^{-1}(i\alpha\hat{u} + D\hat{v}) = \mathcal{C}_\kappa\mathcal{L}_c\hat{T} + \mathcal{C}_d \Big[2\bar{\mu}\bar{U}'(D\hat{u} + i\alpha\hat{v}) + \frac{d\mu}{dT}(\bar{U}')^2\hat{T} \Big], \quad (20)$$

$$\mathcal{L}_c = D^2 - \alpha^2 + \frac{1}{\bar{\kappa}}\frac{d\kappa}{dT}(2\bar{T}'D + \bar{T}'') + \frac{1}{\bar{\kappa}}\frac{d^2\kappa}{dT^2}(\bar{T}')^2, \quad \mathcal{C}_\kappa = \frac{\gamma\bar{\mu}}{Pr Re \bar{\rho}}, \quad \mathcal{C}_d = \frac{\gamma(\gamma - 1)M_e^2}{Re \bar{\rho}}. \quad (21)$$

Reading the equations.

- $i\alpha(\bar{U} - c)$ is *convection by the mean relative to the wave speed*; it vanishes at the **critical layer** $\bar{U} = c_r$, where the inviscid form of the equations becomes singular — which is why that layer governs much of the instability physics.
- D, D^2 are wall-normal derivatives; $-\alpha^2$ comes from $\partial_x^2 \mapsto -\alpha^2$.
- the $\nu_R = 1/(Re\bar{\rho})$ terms are *viscous* (the $\frac{4}{3}, \frac{1}{3}$ are Stokes's bulk viscosity; §9 generalises this to a ratio $\lambda/\mu = 1.2$).
- the $d\mu/dT, d\kappa/dT$ terms are *temperature-dependent transport* — essential at hypersonic speeds.
- $\mathcal{C}_\kappa\mathcal{L}_c\hat{T}$ is heat conduction; $\mathcal{C}_d[\dots]$ is viscous dissipation.

Part IV — Turning equations into matrices

Part III left four coupled ODEs in y with known base-flow coefficients but no closed-form solution. Part IV converts them into a finite matrix a computer can handle.

6 Chebyshev collocation, from scratch

What problem is solved here. Replace the wall-normal derivative D by a finite matrix so the disturbance ODEs become linear algebra; each continuous amplitude $\hat{q}(y)$ becomes a vector of its values at chosen grid points.

Why Chebyshev? For smooth functions a polynomial interpolant through Chebyshev points converges *exponentially*, so few points give many digits. The Chebyshev–Gauss–Lobatto (GLL) nodes on $[-1, 1]$,

$$\xi_j = \cos\left(\frac{\pi j}{N}\right), \quad j = 0, 1, \dots, N, \quad (22)$$

cluster near the ends $\xi = \pm 1$. **Node ordering matters:** $\xi_0 = +1$ and $\xi_N = -1$, so j runs from $+1$ down to -1 .

The differentiation matrix. One polynomial of degree $\leq N$ passes through the values $\{f(\xi_j)\}$; differentiating and evaluating at the nodes is linear, $f'(\xi_j) \approx \sum_k D_{jk} f(\xi_k)$, with (Don & Solomonoff; `spectral.py:chebyshev_D`)

$$D_{jk} = \frac{c_j}{c_k} \frac{(-1)^{j+k}}{\xi_j - \xi_k} \quad (j \neq k), \quad D_{jj} = -\sum_{k \neq j} D_{jk}, \quad c_0 = c_N = 2, \quad c_{\text{else}} = 1. \quad (23)$$

The diagonal is set so each row sums to zero (a constant function has zero slope); $c_0 = c_N = 2$ accounts for the doubled weight of the two endpoints.

In plain words. *D turns “the list of function values” into “the list of slope values.” It is dense — every node’s slope uses every other node’s value — which is why the final stability matrices are dense.*

A 3-point worked example. For $N = 2$, $\xi = (1, 0, -1)$ and (23) gives

$$D = \begin{bmatrix} 1.5 & -2 & 0.5 \\ 0.5 & 0 & -0.5 \\ -0.5 & 2 & -1.5 \end{bmatrix}, \quad f'(\xi_1) \approx \frac{1}{2}f(\xi_0) - \frac{1}{2}f(\xi_2) \text{ (centred difference)}. \quad (24)$$

The second derivative on $[-1, 1]$ is the matrix square, $D_\xi^2 = D_\xi D_\xi$ (*not* the physical D_1 squared — see below).

Mapping to the physical layer. The flow lives on $y \in [0, y_{\max}]$ with points clustered at the *wall*. The algebraic map (Malik 1990; `spectral.py:map_domain`) is

$$y(\xi) = L \frac{1 + \xi}{1 - \xi + 2L/y_{\max}}, \quad L = \frac{y_{\max}}{6} \text{ (stretching parameter)}. \quad (25)$$

This sends $\xi = +1$ (node $j=0$) to $y = y_{\max}$ (**freestream**) and $\xi = -1$ (node $j=N$) to $y = 0$ (**wall**); the grid median sits at $y(\xi=0) = y_{\max}/8$ and roughly half the points lie below $y \approx L$. By the chain rule, with $\xi_y = 1/y_\xi$ and $\xi_{yy} = -y_{\xi\xi}/y_\xi^3$, the physical operators are

$$D_1 = \text{diag}(\xi_y) D_\xi, \quad D_2 = \text{diag}(\xi_y^2) D_\xi^2 + \text{diag}(\xi_{yy}) D_\xi. \quad (26)$$

Common pitfall. The chain rule is applied to the *computational* second-derivative matrix $D_\xi^2 = D_\xi D_\xi$, then mapped by (26). The physical D_2 is *not* $D_1 D_1$.

The physical domain must reach well past the boundary layer, so the truncation is set as a *multiple of the boundary-layer scale*, $y_{\max} = m(\delta^*/L^*)$, with $m \approx 8$ –12 for the wall-trapped second mode and larger ($m \approx 35$ –45) for the more diffuse first mode; the bare values $y_{\max} = 6/12$ are only a fallback. The default resolution is $N = 128$ ($n=N+1=129$); N is a free accuracy parameter, and because the method is spectral the eigenvalue converges *exponentially* in N (Figure 7).

7 Assembling the operator and its boundary conditions

What problem is solved here. Stack the four nodal-value vectors into one length- $4n$ unknown and turn Eqs. (17)–(20) into one $4n \times 4n$ matrix (two matrices, for the generalised problem). Then overwrite the wall/freestream rows with boundary conditions.

Chebyshev–Gauss–Lobatto grid and the wall-clustering map

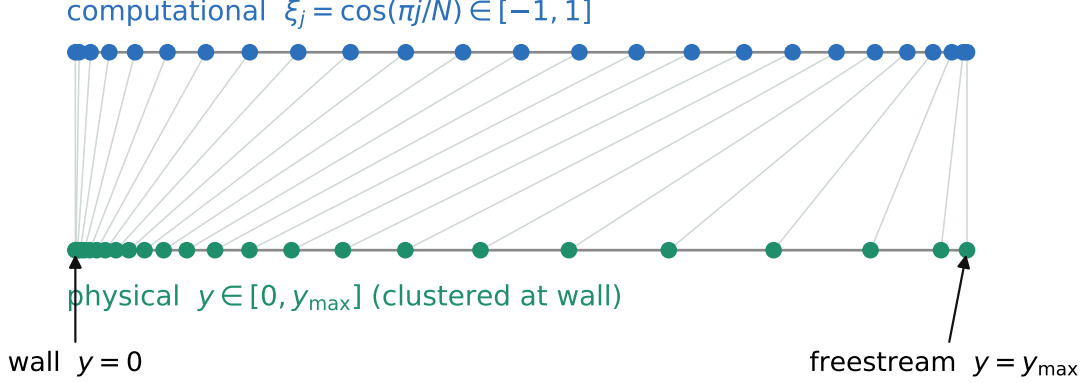


Figure 6: The GLL nodes $\xi_j = \cos(\pi j/N)$ on $[-1, 1]$ (top) map by Eq. (25) to the physical grid $y \in [0, y_{\max}]$ (bottom), piling up at the wall ($\xi = -1$) where eigenfunctions vary fastest.

Each amplitude becomes a length- n vector of nodal values; the four are stacked, $\phi = [\hat{u}; \hat{v}; \hat{T}; \hat{p}] \in \mathbb{C}^{4n}$. In every equation $D \rightarrow D_1$, $D^2 \rightarrow D_2$, and each mean coefficient becomes a diagonal matrix (e.g. $\bar{U} \rightarrow \text{diag}(\bar{U}(y_j))$). Each of the four equations contributes one block-row of four $n \times n$ blocks; the operator is the 4×4 array of blocks (Figure 8).

The construction recipe (how a term becomes a matrix entry). Turning an equation into blocks is mechanical: (i) every mean coefficient $f(y)$ becomes the diagonal $\text{diag}(f(y_j))$; (ii) $D \rightarrow D_1$ and $D^2 \rightarrow D_2$, so a term $f D$ becomes $\text{diag}(f) D_1$ (coefficient times derivative matrix, in that order); (iii) the constants $i\alpha$ (from ∂_x) and $-\alpha^2$ (from ∂_x^2) multiply through; (iv) the result is deposited in the block at (its equation's row, its field's column); (v) anything carrying the eigenvalue c is stripped of c and placed in B at the same address — everything else stays in A . Four equations $\times$ four fields = a 4×4 grid of $n \times n$ blocks per matrix.

The eigenvalue split (which terms go where). For the temporal problem (§8) the only place c appears is the factor $i\alpha(\bar{U} - c)$. Writing each such factor as $i\alpha\bar{U} - c(i\alpha)$ splits the operator: the $i\alpha\bar{U}$ part stays in A , the c -coefficient $-i\alpha$ goes into B . Equivalently $B = -\partial(\text{operator})/\partial c$. The result is that **A is the full block matrix whose rows are exactly Eqs. (17)–(20)** (coefficients moved to one side), and B has only five non-zero blocks:

$$B_{(C,\hat{T})} = -\frac{i\alpha}{\bar{T}}, \quad B_{(C,\hat{p})} = i\alpha\gamma M_e^2, \quad B_{(X,\hat{u})} = i\alpha, \quad B_{(Y,\hat{v})} = i\alpha, \quad B_{(E,\hat{T})} = i\alpha. \quad (27)$$

Worked example — the continuity block-row, built term by term. Apply the recipe to Eq. (17), splitting the convective factor $i\alpha(\bar{U} - c) = i\alpha\bar{U} - c i\alpha$ so the $i\alpha\bar{U}$ part stays in A and the c -coefficient $i\alpha$ goes to B . Reading off the four fields:

$$\begin{aligned} A_{(C,\hat{u})} &= i\alpha I, & A_{(C,\hat{v})} &= D_1 - \text{diag}(\bar{T}'/\bar{T}), \\ A_{(C,\hat{p})} &= i\alpha\gamma M_e^2 \text{diag}(\bar{U}), & A_{(C,\hat{T})} &= -i\alpha \text{diag}(\bar{U}/\bar{T}), \\ B_{(C,\hat{p})} &= i\alpha\gamma M_e^2 I, & B_{(C,\hat{T})} &= -i\alpha \text{diag}(1/\bar{T}). \end{aligned}$$

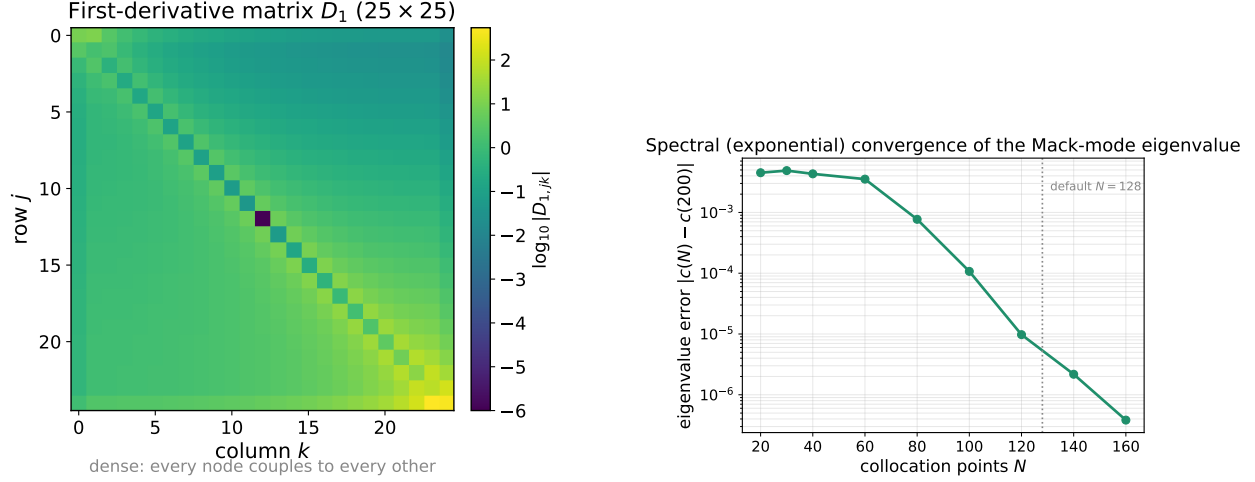


Figure 7: *Left:* the physical first-derivative matrix D_1 (25×25 here) is dense ($\log_{10}$ of the entry magnitudes shown); spectral accuracy couples every node to every other. *Right:* the payoff — the worked-example eigenvalue error $|c(N) - c(200)|$ falls *exponentially* with N (a straight line on this semilog axis), so the default $N = 128$ already gives many digits.

Six non-zero entries — four in A , two in B . The eigenvalue here multiplies the state-law $\hat{p}, \hat{T}$ terms (not a convective velocity), which is why row C contributes *two* B -blocks while the momentum/energy rows each contribute one. Every other equation is processed identically.

All sixteen A -blocks (complete listing, transcribed from `pymack/temporal_solver.py`). Rows are the four equations, columns the four fields ($\hat{u}, \hat{v}, \hat{T}, \hat{p}$); $\nu_R = 1/(Re\bar{\rho})$, with $\mathcal{C}_\kappa, \mathcal{C}_d, \mathcal{L}_c$ as defined under Eq. (20), and a bare mean symbol denotes its diagonal matrix. The five B -blocks are Eq. (27); *every block not listed is zero* — notably $A_{(E,\hat{p})} = 0$, since the energy row has no pressure term.

$$(C) \quad \hat{u} \rightarrow i\alpha I; \quad \hat{v} \rightarrow D_1 - \bar{T}'/\bar{T}; \quad \hat{T} \rightarrow -i\alpha\bar{U}/\bar{T}; \quad \hat{p} \rightarrow i\alpha\gamma M_c^2\bar{U}.$$

$$(X) \quad \hat{u} \rightarrow i\alpha\bar{U} - \nu_R(\bar{\mu}D_2 + \bar{\mu}'D_1) + \frac{4}{3}\alpha^2\nu_R\bar{\mu}; \quad \hat{v} \rightarrow \bar{U}' - i\alpha\nu_R(\frac{1}{3}\bar{\mu}D_1 + \bar{\mu}'); \quad \hat{T} \rightarrow -\nu_R[\frac{d\mu}{dT}(\bar{U}'D_1 + \bar{U}'') + \frac{d^2\mu}{dT^2}\bar{U}'\bar{T}']; \\ \hat{p} \rightarrow i\alpha\bar{\rho}^{-1}.$$

$$(Y) \quad \hat{u} \rightarrow -i\alpha\nu_R(\frac{1}{3}\bar{\mu}D_1 - \frac{2}{3}\bar{\mu}'); \quad \hat{v} \rightarrow i\alpha\bar{U} - \nu_R(\frac{4}{3}\bar{\mu}D_2 + \frac{4}{3}\bar{\mu}'D_1) + \alpha^2\nu_R\bar{\mu}; \quad \hat{T} \rightarrow -i\alpha\nu_R\frac{d\mu}{dT}\bar{U}'; \quad \hat{p} \rightarrow \bar{\rho}^{-1}D_1.$$

$$(E) \quad \hat{u} \rightarrow i\alpha(\gamma - 1)\bar{\rho}^{-1} - 2\mathcal{C}_d\bar{\mu}\bar{U}'D_1; \quad \hat{v} \rightarrow \bar{T}' + (\gamma - 1)\bar{\rho}^{-1}D_1 - 2i\alpha\mathcal{C}_d\bar{\mu}\bar{U}'; \quad \hat{T} \rightarrow i\alpha\bar{U} - \mathcal{C}_\kappa\mathcal{L}_c - \mathcal{C}_d\frac{d\mu}{dT}(\bar{U}')^2; \quad \hat{p} \rightarrow 0.$$

Holding these against `temporal_solver.py` line for line: row C is `A[blk(0,.)]`, row X `A[blk(1,.)]`, and so on; the diagonals $\bar{U}, \bar{\mu}, \dots$ are the `np.diag(...)` arrays, D_1, D_2 the differentiation matrices, and $\nu_R = \text{rhob_inv}/\text{Re}$.

Boundary conditions = row replacement, with explicit indices. A collocation row says “the equation holds at node y_j .” At the wall and freestream we overwrite those rows. With the stacking $\phi = [\hat{u}; \hat{v}; \hat{T}; \hat{p}]$, field block $b \in \{0, 1, 2, 3\}$ occupies global indices $bn, \dots, bn + n - 1$, and (from the ordering above) local index 0 is the freestream, local index $n-1$ the wall. Hence:

$A\phi = cB\phi$: each $4n \times 4n$ matrix is a 4×4 grid of $n \times n$ blocks (rows = equations, columns = fields)

A — dense operator (15 blocks)					B — the c -terms only (5 blocks)				
	$\hat{u}$	$\hat{v}$	$\hat{T}$	$\hat{p}$		$\hat{u}$	$\hat{v}$	$\hat{T}$	$\hat{p}$
continuity + state	ial	$D_1 - \frac{\hat{p}}{\hat{T}}$	$-ia\frac{\hat{u}}{\hat{T}}$	$ia\gamma M^2 \hat{U}$	continuity + state			$-\frac{ia}{\hat{T}}$	$ia\gamma M^2$
x-momentum	$ia\hat{U}$ + visc.	$\hat{U}'$ + visc.	transport	$ia\hat{p}^{-1}$	x-momentum	ia			
y-momentum	visc.	$ia\hat{U}$ + visc.	transport	$\hat{p}^{-1}D_1$	y-momentum		ia		
energy	dissip.	$\hat{T}' + \dots$	$ia\hat{U}$ $- c_k \mathcal{E}_c$	0	energy			ia	

Figure 8: Block structure of $A\phi = cB\phi$. Rows = the four equations (continuity, x -mom, y -mom, energy); columns = the four fields $\hat{u}, \hat{v}, \hat{T}, \hat{p}$. Shaded = non-zero block. A (left) is block-dense; B (right) holds only the five blocks of Eq. (27) — every term carrying the eigenvalue c .

condition	field	global row	row becomes
no-slip (wall)	$\hat{u}$	$n - 1$	identity ($\hat{u} = 0$)
no-slip (wall)	$\hat{v}$	$2n - 1$	identity ($\hat{v} = 0$)
thermal (wall)	$\hat{T}$	$3n - 1$	identity ($\hat{T} = 0$, isothermal) or $D_1[\text{wall}, :]$ ($D\hat{T} = 0$, adiabatic)
decay (freestream)	$\hat{u}$	0	identity
decay (freestream)	$\hat{v}$	n	identity
decay (freestream)	$\hat{T}$	$2n$	identity

The same rows of B are zeroed so the constraint is algebraic; $\hat{p}$ carries no explicit boundary row.

Getting this backwards silently fails. Because $\xi_0 = +1 \rightarrow y_{\max}$ and $\xi_N = -1 \rightarrow y = 0$, node index 0 is the freestream and index $n-1$ is the wall — the opposite of the naive guess. Swapping them produces plausible-looking but wrong eigenvalues.

Part V — The eigenvalue problems

The matrices are built, so “does the wave grow?” is now finite linear algebra. There are two equivalent ways to pose it, depending on which of α, ω we hold real.

8 Temporal problem: a generalised eigenvalue problem

What problem is solved here. Given a real wavenumber α , find the complex phase speed c and shape ϕ . **Equation:** the four disturbance equations. **Eigenvalue:** c . **Growth:** $\omega_i = \alpha \text{Im}(c)$.

Boundary conditions = row replacement
in the state vector $[\hat{u}; \hat{v}; \hat{T}; \hat{p}]$

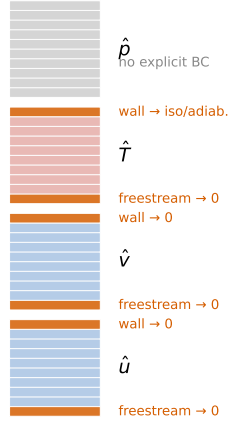


Figure 9: The boundary conditions as row replacement: the stacked state $[\hat{u}; \hat{v}; \hat{T}; \hat{p}]$ with the six overwritten rows highlighted (wall and freestream for $\hat{u}, \hat{v}, \hat{T}$); $\hat{p}$ carries no boundary row. Note index 0 is the freestream and $n-1$ the wall.

With A, B from §7 this is a **generalised eigenvalue problem**

$$A\phi = cB\phi, \quad A, B \in \mathbb{C}^{4n \times 4n}, \quad (28)$$

solved directly by the QZ algorithm (`scipy.linalg.eig`), which returns all $4n$ eigenvalues at once.

In plain words. An ordinary eigenproblem is $A\phi = c\phi$; a generalised one allows a second matrix B on the right. The discrete eigenvector ϕ is just the eigenfunction $\hat{q}(y)$ sampled at the grid points — “eigenfunction” and “eigenvector” are the continuous and discrete names for the same shape.

A disturbance $\propto e^{i\alpha(x-ct)}$ with α real behaves as $e^{\alpha c_i t}$ in time, so

$$c_i = \text{Im}(c) > 0 \Rightarrow \text{unstable (grows in time)}, \quad c_r = \text{Re}(c) = \text{wave speed}. \quad (29)$$

The wave speed classifies the mode: a first (viscous/Tollmien–Schlichting) mode has $c_r \approx 0.3\text{--}0.6$; a second (Mack) mode has $c_r \approx 1 - 1/M_e$. The raw spectrum is filtered to $\text{Re}(c) \in (-0.5, 1.5)$, $|\text{Im}(c)| < 0.5$ and sorted by c_i .

9 Spatial problem: a quadratic eigenvalue problem

What problem is solved here. Given a real frequency ω , find the complex wavenumber α and shape. **Equation:** the same physics grouped by powers of α . **Eigenvalue:** α . **Growth:** $\sigma = -\text{Im}(\alpha)$.

A real source forces a fixed frequency and the disturbance grows in *space*. Now ω is real and α is the unknown; because viscous/conduction terms carry $\partial_x^2 \mapsto -\alpha^2$, α appears *quadratically*. Grouping the discretised Eqs. (17)–(20) by powers of α (substitute $D \rightarrow D_1$, $D^2 \rightarrow D_2$, coefficients $\rightarrow$ diagonals, then collect $\alpha^0, \alpha^1, \alpha^2$) gives a **quadratic eigenvalue problem** (QEP)

$$(C_0 + \alpha C_1 + \alpha^2 C_2)\phi = 0. \quad (30)$$

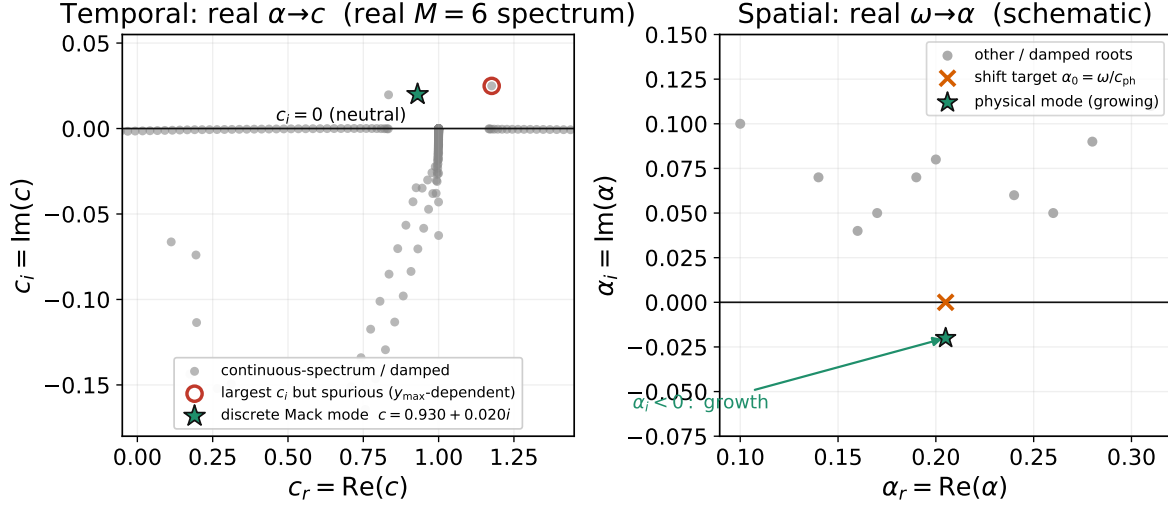


Figure 10: Where the eigenvalues live. *Left (temporal):* input real α ; the phase speeds c populate the complex plane; the physical discrete mode sits above the line $c_i = \text{Im}(c) = 0$ (unstable) while the continuous spectrum is damped. *Right (spatial, explained in §9):* input real ω ; the wavenumbers α are sought near a shift target α_0 ; the physical mode has $\alpha_i < 0$, i.e. growth.

Opening up C_0, C_1, C_2 (complete listing). The spatial assembler (`equations.py`) uses the *enthalpy* energy form — so $\hat{p}$ appears explicitly via Dp'/Dt and the temperature equation differs from (20) — and a bulk-viscosity ratio $\lambda/\mu = 1.2$. The resulting viscous coefficients are

$$c_{\alpha^2} = \frac{2(2+\lambda/\mu)}{3} = 2.13, \quad c_{\times} = \frac{1+2\lambda/\mu}{3} = 1.13, \quad c_a = \frac{2(\lambda/\mu-1)}{3} = 0.13 \quad (\text{Mack Eq. 8.5}). \quad (31)$$

Because this closure is *not* obtained by re-grouping (17)–(20), here are *all* non-zero blocks, transcribed from `equations.py`. Write $\nu_R = 1/(Re\bar{\rho})$ for the viscous/conduction prefactor and $d_R = (\gamma - 1)M_e^2\nu_R$ for the dissipation prefactor; each entry below is the operator acting on the named field in that equation's block-row.

$$\begin{aligned} (C) \quad C_0: \hat{v} &\rightarrow D_1 - \frac{\bar{T}'}{\bar{T}}, \quad \hat{T} \rightarrow \frac{i\omega}{\bar{T}}, \quad \hat{p} \rightarrow -i\omega\gamma M_e^2, \quad C_1: \hat{u} \rightarrow iI, \quad \hat{T} \rightarrow -\frac{i\bar{U}'}{\bar{T}}, \quad \hat{p} \rightarrow i\gamma M_e^2\bar{U}, \quad C_2: \text{none.} \\ (X) \quad C_0: \hat{u} &\rightarrow -i\omega - \nu_R(\bar{\mu}D_2 + \bar{\mu}'D_1), \quad \hat{v} \rightarrow \bar{U}', \quad \hat{T} \rightarrow -\nu_R\left[\frac{d\bar{\mu}}{dT}(\bar{U}'' + \bar{U}'D_1) + \frac{d^2\bar{\mu}}{dT^2}\bar{U}'\bar{T}'\right], \quad C_1: \hat{u} \rightarrow i\bar{U}, \quad \hat{v} \rightarrow -i\nu_R(c_{\times}\bar{\mu}D_1 + \bar{\mu}'), \quad \hat{p} \rightarrow i\bar{\rho}^{-1}, \quad C_2: \hat{u} \rightarrow c_{\alpha^2}\nu_R\bar{\mu}. \\ (Y) \quad C_0: \hat{v} &\rightarrow -i\omega - c_{\alpha^2}\nu_R(\bar{\mu}D_2 + \bar{\mu}'D_1), \quad \hat{p} \rightarrow \bar{\rho}^{-1}D_1, \quad C_1: \hat{u} \rightarrow -i\nu_R(c_{\times}\bar{\mu}D_1 + c_a\bar{\mu}'), \quad \hat{v} \rightarrow i\bar{U}, \quad \hat{T} \rightarrow -i\nu_R\frac{d\bar{\mu}}{dT}\bar{U}', \quad C_2: \hat{v} \rightarrow \nu_R\bar{\mu}. \\ (E) \quad C_0: \hat{u} &\rightarrow -2d_R\bar{\mu}\bar{U}'D_1, \quad \hat{v} \rightarrow \bar{T}', \quad \hat{T} \rightarrow -i\omega - \nu_R[\bar{\kappa}D_2 + 2\bar{\kappa}'D_1 + \frac{d\bar{\kappa}}{dT}\bar{T}'' + \frac{d^2\bar{\kappa}}{dT^2}(\bar{T}')^2] - d_R\frac{d\bar{\mu}}{dT}(\bar{U}')^2, \quad \hat{p} \rightarrow i\omega(\gamma - 1)M_e^2\bar{\rho}^{-1}, \quad C_1: \hat{v} \rightarrow -2id_R\bar{\mu}\bar{U}', \quad \hat{T} \rightarrow i\bar{U}, \quad \hat{p} \rightarrow -i(\gamma - 1)M_e^2\bar{U}\bar{\rho}^{-1}, \quad C_2: \hat{T} \rightarrow \nu_R\bar{\kappa}. \end{aligned}$$

All other blocks are zero. The temporal closure (17)–(20) (Stokes $\frac{4}{3}, \frac{1}{3}$ factors, temperature energy row) and this spatial closure (enthalpy row, $\lambda/\mu = 1.2$) differ only in the bulk-viscosity treatment and the energy-equation arrangement; both approximate the same physical mode and agree on it to within discretisation error.

Companion linearisation. A QEP becomes an ordinary (generalised) problem of *twice* the size by introducing $\psi = \alpha\phi$ (`solver.py`):

$$\underbrace{\begin{bmatrix} -C_1 & -C_0 \\ I & 0 \end{bmatrix}}_{\mathcal{L}} \begin{bmatrix} \psi \\ \phi \end{bmatrix} = \alpha \underbrace{\begin{bmatrix} C_2 & 0 \\ 0 & I \end{bmatrix}}_{\mathcal{R}} \begin{bmatrix} \psi \\ \phi \end{bmatrix}. \quad (32)$$

The lower block row enforces $\psi = \alpha\phi$; the upper row then reproduces (30). The eigenvalue is α and the physical shape is the *lower* half ϕ (Figure 11).

Why “the lower half”. Since $\psi = \alpha\phi$ the two halves are parallel, so both satisfy the QEP up to scaling; by convention ϕ (lower half) is the one normalised as the eigenfunction.

Companion linearisation: a quadratic EVP becomes an ordinary EVP of double size

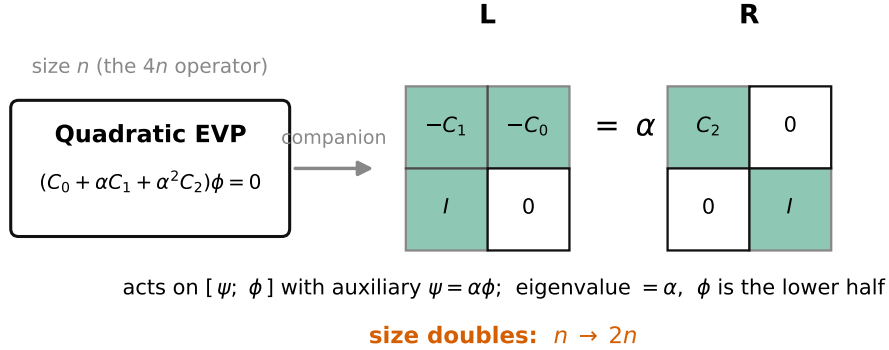


Figure 11: Companion linearisation (32): the $4n$ -sized quadratic problem becomes an ordinary generalised problem of size $8n$ in $[\psi; \phi]$, with $\psi = \alpha\phi$. Its eigenvalue is exactly α ; the eigenfunction is the lower half.

Shift–invert (which eigenvalue, cheaply). We target the relevant branch with a shift $\alpha_0 = \omega/c_{\text{ph}}$ ($c_{\text{ph}} = 1 - 1/M_e$ for $M_e > 3$, else ≈ 0.4). Subtracting α_0 and inverting maps eigenvalues *near* α_0 to *large* μ , where

$$\text{eigenvalues of } (\mathcal{L} - \alpha_0 \mathcal{R})^{-1} \mathcal{R} \text{ are } \mu = \frac{1}{\alpha - \alpha_0}, \quad \Rightarrow \quad \alpha = \alpha_0 + \frac{1}{\mu}. \quad (33)$$

So returning the largest- $|\mu|$ modes returns the wavenumbers nearest the physical one (one LU factorisation; in `pyMack` a dense solve followed by selecting the $n_{\text{modes}}=20$ nearest α_0 , then sorting by $\text{Im}(\alpha)$). The amplitude $\propto e^{i\alpha x} = e^{i\alpha_r x} e^{-\alpha_i x}$, so

$$\sigma = -\text{Im}(\alpha) = -\alpha_i > 0 \Rightarrow \text{unstable (grows downstream)}. \quad (34)$$

Alternative route — Newton with a phase-speed seed. A more robust path when shift–invert struggles (`solve_spatial_from_temporal`): solve the *temporal* problem at a real trial $\alpha_r = \omega/c_{\text{est}}$ ($c_{\text{est}} = 0.98$ for $M_e > 3$, else 0.35), keeping only modes that agree across two resolutions (tolerance 8×10^{-3}); form the seed $\text{Im}(\alpha) \approx -\alpha_r c_i / c_r$ (a *phase-speed* estimate — the rigorous Gaster relation uses the group velocity $c_g = d\omega_r / d\alpha_r$, but here it only seeds Newton, so precision is unneeded); then Newton-iterate the dispersion residual $F(\alpha) = \alpha c(\alpha) - \omega = 0$, $\alpha^{(k+1)} = \alpha^{(k)} - F/F'$, with $F' = c + \alpha dc/d\alpha$ approximated by a finite difference (step 10^{-5} , tol 10^{-6} , ≤ 20 iterations). Both routes return the same $\sigma = -\text{Im}(\alpha)$.

10 Which eigenvalue is physical?

What problem is solved here. Given the full $4n$ -element spectrum, identify the one eigenvalue that is a genuine boundary-layer mode rather than continuous-spectrum noise.

Besides the discrete boundary-layer modes the operator has a dense **continuous spectrum** — on an infinite domain these freestream disturbances form an unbroken band rather than isolated points, and the finite grid renders them as a dense cluster. Two tests (`discrete_mode.py`) keep only the physical mode (Figure 12):

1. **Freestream decay:** a true boundary-layer eigenfunction decays as $y \rightarrow y_{\max}$, a continuous-spectrum one does not. Reject candidates whose edge-to-peak amplitude ratio exceeds ~ 0.06 .
2. **Domain stationarity:** a genuine eigenvalue barely moves if the truncation changes. Recompute at two domain heights and keep modes whose c matches to $\sim 4 \times 10^{-3}$; accept growth rates with $|c_i| \lesssim 0.05$ and c_r in the physical band.

In plain words. *These two tests are the difference between a clean growth rate and noise — they are what let us trust a single number out of the full $4n$ -element spectrum (724 modes in the worked example of §11, where the largest-growth eigenvalue is in fact spurious).*

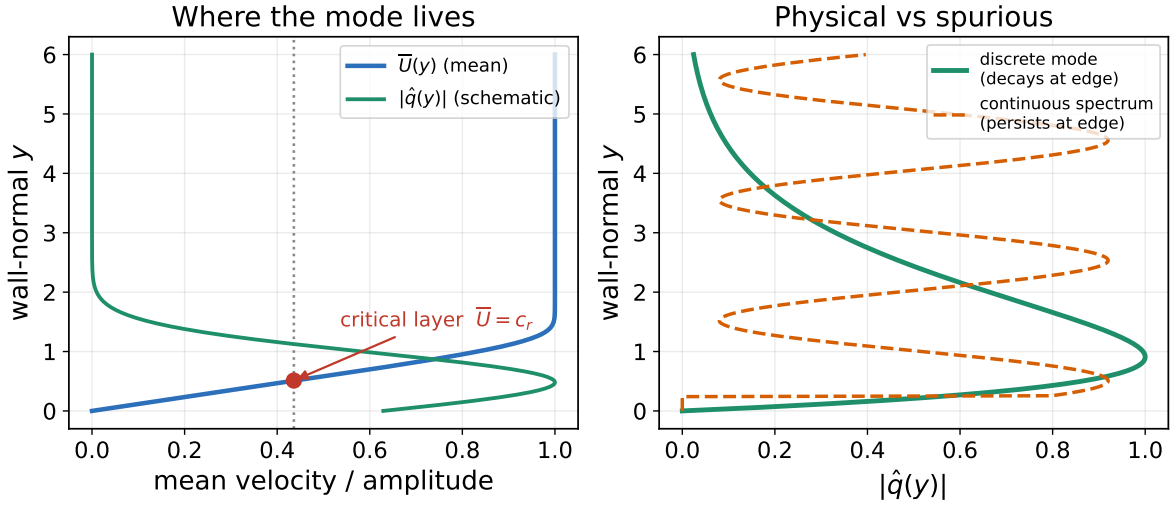


Figure 12: *Left:* the critical layer is where $\bar{U}(y) = c_r$; the mode’s amplitude $|\hat{q}(y)|$ tends to peak near it. *Right:* a discrete boundary-layer mode (solid) decays toward the freestream, while a continuous-spectrum mode (dashed) keeps oscillating to the edge — the basis for the two acceptance tests.

11 Worked example: one temporal solve, start to finish

Take a strong Mack-mode case: $M_e = 6.0$, $R = 5500$, $\alpha_L = 0.174$, adiabatic wall, L^* scaling, $N = 180$ ($n = 181$, so the matrices are $4n = 724$ wide). The wall-normal domain is $y_{\max} = 10(\delta^*/L^*) \approx 128$ (§6: the domain is a multiple of the boundary-layer scale, not a bare number). Solving (28) returns 724 eigenvalues; after the physical filter the six least-stable are

c_r	c_i	$\omega_i = \alpha c_i$	$y_{\max}$ -stationary?	verdict
+1.176	+0.0249	+0.00434	no ($ \Delta c \sim 10^{-2}$)	spurious / continuous spectrum
+0.930	+0.0200	+0.00348	yes ($ \Delta c \sim 10^{-7}$)	discrete Mack mode — the answer
+0.834	+0.0197	+0.00343	no	continuous spectrum
+0.740	+0.0000	0.00000	—	neutral continuous-spectrum cluster
+0.720	+0.0000	0.00000	—	continuous spectrum
+0.760	+0.0000	0.00000	yes	stable discrete mode

This is the trap the mode tests of §10 exist to avoid: the *largest*-growth eigenvalue (row 1, $c_r = 1.18$) is *not* the answer — it is spurious, its c shifting by $\sim 10^{-2}$ when the domain height changes (and $c_r > 1$ is unphysically supersonic). The physical answer is row 2, the **Mack (second) mode** $c = 0.930 + 0.0200i$: domain-stationary to 10^{-7} , with an eigenfunction that decays at the freestream (Figure 13). Its growth rate is $\omega_i = \alpha c_i = +0.00348$ (unstable) and its phase speed $c_r = 0.93$ is the second-mode value explained in §12.

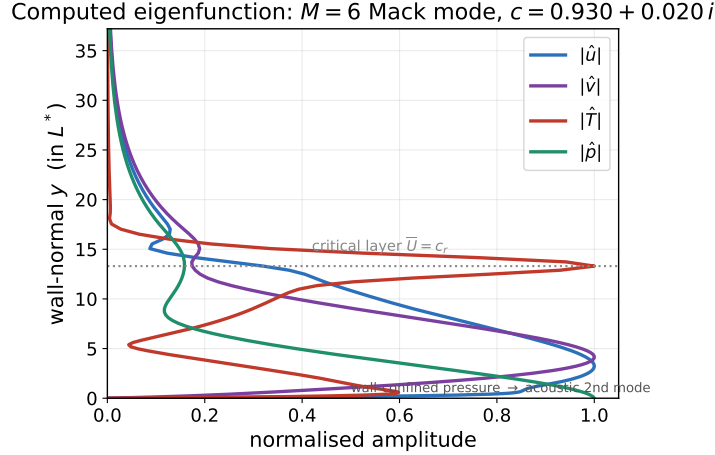


Figure 13: Computed eigenfunction of the worked-example Mack mode: the four amplitude fields $|\hat{u}|, |\hat{v}|, |\hat{T}|, |\hat{p}|$ vs y (each normalised). $|\hat{T}|$ spikes at the critical layer ($\bar{U} = c_r$); $|\hat{p}|$ is wall-confined — the acoustic signature of the second mode (§12). All fields decay at the freestream, confirming a discrete mode.

This single $\omega_i > 0$ is one point “inside” the neutral curve: repeating the solve across an (R, α) grid and finding where ω_i crosses zero traces the neutral curve of Part VI. (All numbers reproduce from `solve_temporal_2d`.)

12 What the modes *are*: two instability mechanisms

We have a growth rate; now the *physics* behind the mode labels, so that $c = 0.930 + 0.0200i$ becomes an understood wave, not a number.

First (Tollmien–Schlichting) mode. At *low* speed the first mode is a viscous instability ($c_r \approx 0.3\text{--}0.6$): counter-intuitively, viscosity introduces a phase lag between u' and v' near the wall, so the Reynolds stress $-\bar{\rho} \overline{u'v'}$ no longer averages to zero and draws energy from the mean shear. As the Mach number rises, compressibility creates a generalised inflection point (below) and the first mode becomes increasingly *inviscid/inflectional* — governed by that point, with viscosity then often *stabilising*. It is the dominant instability only up to $Me \approx 4$.

Second (Mack) mode — a trapped acoustic wave. Above $Me \approx 4$ a second, usually stronger, instability appears: it is essentially *a sound wave trapped between the wall and the “relative sonic line”*, reflecting in a near-wall acoustic cavity (Figure 14). This one picture explains three facts we earlier had to accept blindly:

- its phase speed is $c_r \approx 1 - 1/Me$ — the *limiting* “edge-sonic” value at which the wave is sonic relative to the edge flow, which is why the shift target $\alpha_0 = \omega/(1 - 1/Me)$ of §9 works; the

realised c_r sits somewhat above this and varies along the branch (0.93 vs $1 - 1/6 = 0.83$ here), so it is a guide, not an exact value;

- it is *frequency-tuned*: like any resonator its wavelength is set by the cavity thickness, so the most-amplified wavelength is $\sim 2\delta$, giving the order-of-magnitude scaling $f \sim U_e/2\delta$ — the second-mode frequency rises as the layer thins downstream;
- its eigenfunction (Figure 13) has a *wall-peaked pressure* — the acoustic field in the cavity.

The organising quantity: relative Mach number. Both pictures live in the **relative Mach number** $M_{\text{rel}}(y) = (\bar{U} - c_r)/\bar{a}(y)$, with $\bar{a} = \sqrt{\bar{T}}/M_e$ the local sound speed. The critical layer is $M_{\text{rel}} = 0$; the relative sonic line is $|M_{\text{rel}}| = 1$; the second-mode acoustic cavity is the near-wall region where $|M_{\text{rel}}| > 1$ (Figure 14).

Why a compressible layer is unstable at all (the generalised inflection point). Rayleigh’s theorem: an inviscid *incompressible* shear flow needs a velocity inflection point ($\bar{U}'' = 0$) to be unstable. Compressibility generalises this (Lees–Lin) to the **generalised inflection point** where $\frac{d}{dy}(\bar{\rho}\bar{U}') = 0$ — the necessary condition for the inviscid *first* mode. Aerodynamic heating puts a hot, low-density layer near the wall (the $\bar{T}$ bulge in Figure 4, where the point is marked) and pushes that generalised inflection point up into the layer — giving the robust inviscid first-mode instability that low-speed layers lack. The *second* (Mack) mode needs no inflection point: its existence condition is instead the near-wall relative-supersonic cavity ($|M_{\text{rel}}| > 1$) above.

The wall-cooling reversal (the key design lever). Cooling the wall *stabilises* the first mode (it weakens the near-wall viscous mechanism) but *destabilises* the second (it thickens the relative-supersonic cavity and raises the tuned frequency). This is why cold-walled hypersonic vehicles are second-mode dominated — and why the solver carries an isothermal-wall option.

Phase vs group velocity. c_r is the *phase* speed (crest speed); energy travels at the *group* velocity $c_g = d\omega_r/d\alpha_r$. It is c_g that rigorously links temporal to spatial growth, which is why the phase-speed Gaster estimate of §9 is only a seed.

A compact comparison places any computed eigenvalue at a glance:

	First (TS) mode	Second (Mack) mode
mechanism	viscous (low speed) $\rightarrow$ inviscid inflectional (high speed)	trapped acoustic wave (wall–sonic line)
phase speed c_r	0.3–0.6	$\approx 1 - 1/M_e$ (here 0.93)
dominant when	low speed, $M_e \lesssim 4$	high speed, $M_e \gtrsim 4$
frequency	broadband	tuned, $f \sim U_e/2\delta$
existence criterion	generalised inflection point (Lees–Lin)	relative-supersonic cavity, $ M_{\text{rel}} > 1$
wall cooling	stabilises	destabilises
most-unstable wave	oblique (3D)	2D ($\psi = 0$)

Mack mode = sound trapped between wall and sonic line ($M_e = 6$)

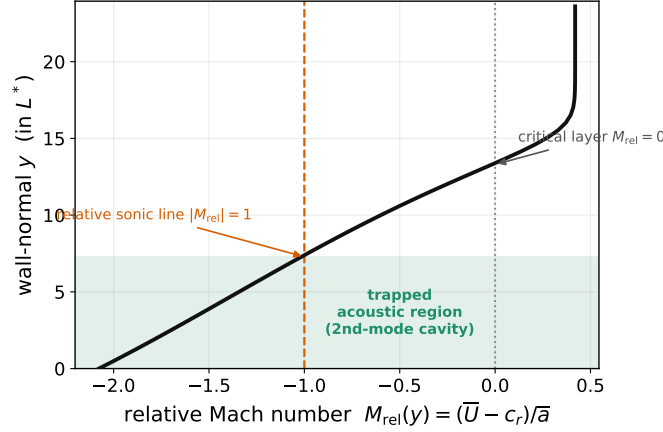


Figure 14: The second-mode mechanism, for the worked-example mode. The relative Mach number $M_{\text{rel}}(y) = (\bar{U} - c_r)/\bar{a}$ is supersonic ($|M_{\text{rel}}| > 1$, shaded) in a near-wall cavity bounded by the wall and the relative sonic line ($|M_{\text{rel}}| = 1$), and passes through zero at the critical layer. The Mack mode is a sound wave trapped in that cavity.

Part VI — Products: neutral curves and N -factors

One solve gives one growth rate. Sweeping the solve over parameters produces the two engineering outputs: the neutral curve (boundary of instability) and the N -factor (a transition predictor).

13 Neutral curves

What problem is solved here. Map the boundary between growth and decay: the locus where the growth rate is exactly zero.

The neutral curve is $c_i(R, \alpha) = 0$ (temporal) or $\sigma(R, \omega) = 0$ (spatial); inside it the flow is unstable. `pyMack` draws it in three steps:

1. **Sweep** an (R, α) grid at fixed M_e ; at each node solve (28), apply the mode tests (§10), store c_i (`build_*_grid.py`). This is the field in Figure 15.
2. **Extract** the zero contour with marching squares (a standard method for tracing a $c_i = 0$ level set through a grid of values; `extract_contours.py`); unresolved nodes are tagged stable so the loop closes.
3. **Continuation** for hard branches: at low Mach the first-mode eigenvalue sinks into the continuous spectrum. `trace_continuation.py` tracks it *by proximity* — from a clean seed it marches in R , follows the nearest c , and secant-solves $c_i = 0$ at each step (continuation = using each converged eigenvalue as the guess for the next nearby parameter value).

14 N -factor and transition

What problem is solved here. Convert local spatial growth into a single number predicting transition.

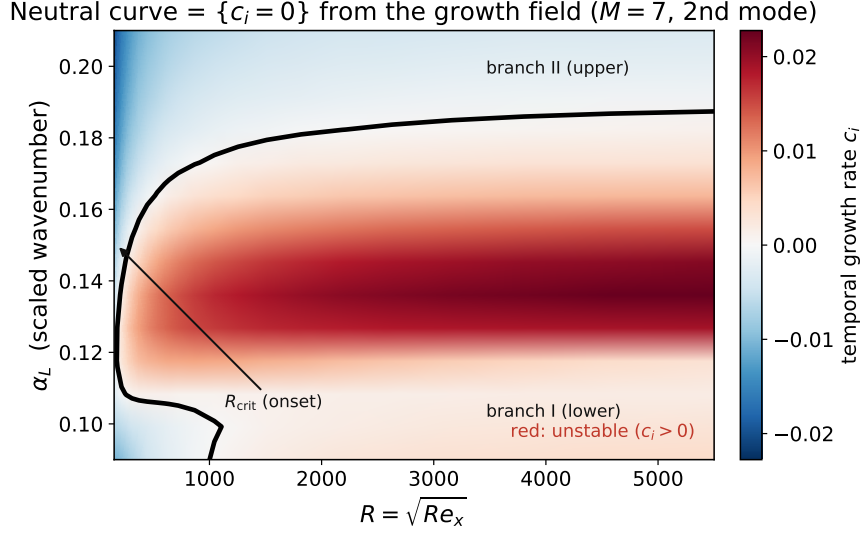


Figure 15: A real growth field $c_i(R, \alpha)$ from a parameter sweep ($M_e = 7$, second mode). Colour is the temporal growth c_i (red = unstable, $c_i > 0$); the thick black curve is the neutral contour $c_i = 0$ that pyMack reports. Axes are the Mack variables $R = \sqrt{Re_x}$ and $\alpha_L = \alpha L^*$.

What matters for transition is the *accumulated* amplification of a fixed-frequency disturbance. For a chosen F , solve the spatial problem (§9) at successive stations to get $\sigma_L = -\text{Im}(\alpha_L)$ — this is the same spatial growth σ of Eq. (34), expressed in the L^* (Mack-length) scaling of Eq. (8) — and integrate from the lower neutral point R_0 (where σ_L first turns positive):

$$N(R) = \int_{R_0}^R \max(\sigma_L, 0) dR' \quad (\text{trapezoidal}), \quad \frac{A(R)}{A(R_0)} = e^{N(R)}. \quad (35)$$

The amplitude grows by e^N ; transition is empirically expected near $N \approx 9\text{--}11$ (the e^N method; `analysis.py`). For a cone, surface arc length is mapped to an equivalent R before integrating.

e^N is a correlation, not a criterion. The transition N is *not* universal: it depends on the disturbance environment (flight $\sim 9\text{--}11$; noisy conventional tunnels can transition at $N \sim 4\text{--}6$). It presumes a single dominant linear mode and stops at the onset of nonlinear breakdown, and it silently absorbs the unknown initial amplitude set by receptivity (§15).

15 Scope and limitations

This document solves the *2D, parallel-flow, modal* eigenvalue problem and converts its growth into an e^N estimate. That is one stage of transition (Figure 2); a careful reader should keep four idealizations in view:

1. **Parallel flow.** Setting $\bar{v} = 0$ and y -only coefficients neglects the $O(1/R)$ streamwise growth of the layer; the standard remedy is the *Parabolised Stability Equations* (PSE), which march the disturbance downstream retaining slow x -variation.
2. **Two-dimensionality.** The solver takes $\beta = 0$. The most-unstable *first* mode is oblique ($\psi \sim 55^\circ$, Squire’s theorem fails under compressibility), so 2D first-mode growth is a *lower bound*; the *second*/Mack mode is most unstable at $\psi = 0$, so the 2D result is its physical maximum.

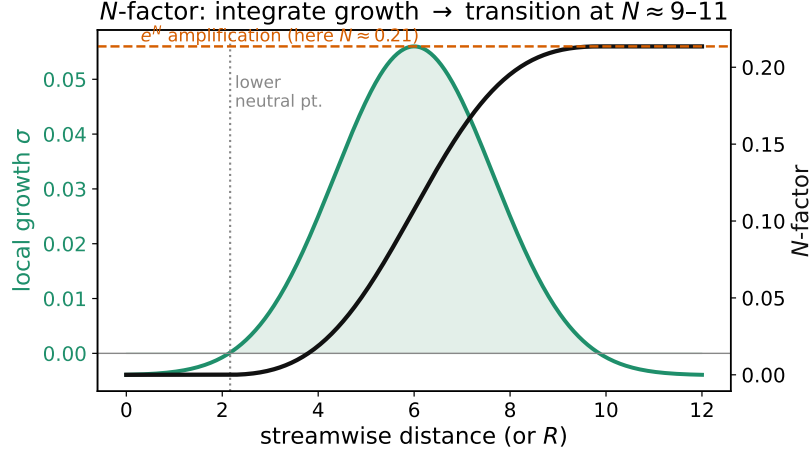


Figure 16: The N -factor. *Shaded/green (left axis):* local spatial growth $\sigma = -\alpha_i$, non-zero only between the two neutral points. *Black (right axis):* its running integral $N(R)$, the exponent of the amplification e^N . Transition correlates with $N \approx 9$ –11.

3. **Modal analysis only.** Eigenvalues describe asymptotic growth; the non-normal operator also permits *transient (algebraic) growth* that modal analysis misses, relevant to bypass and blunt-body paths.
4. **Receptivity and e^N .** LST gives the amplification *ratio* $A(R)/A(R_0) = e^N$, not the initial amplitude $A(R_0)$; setting that is the separate *receptivity* problem, and e^N is an environment-dependent correlation rather than a first-principles criterion.

None of these affect the equations or numerics above; they bound their interpretation. The worked-example second mode (§11) is captured faithfully by the 2D modal solver.

16 Summary and replication checklist

Goal	Problem solved	Method	Growth
Temporal growth, neutral curves	generalised EVP $A\phi = cB\phi$, real α (Eq. 28)	Chebyshev collocation + QZ	αc_i
Spatial growth, N -factor	quadratic EVP $(C_0 + \alpha C_1 + \alpha^2 C_2)\phi = 0$, real ω (Eq. 30)	companion + shift-invert; or temporal + Newton seed	$-\alpha_i$
Neutral curve	$\{c_i = 0\}$ over an (R, α) sweep	grid sweep $\rightarrow$ marching squares; continuation	—
Transition	$N = \int \max(\sigma_L, 0) dR$ (Eq. 35)	fixed- F spatial march + integration	—

To re-implement the solver from scratch.

1. Solve the base-flow boundary-value problem (9)–(12); tabulate $\bar{U}, \bar{T}$ and derivatives.
2. Build ξ_j (22), D_ξ (23), map to y (25), form D_1, D_2 (26).
3. Assemble the 4×4 block operator from Eqs. (17)–(20) (temporal: A, B with the split (27); spatial: C_0, C_1, C_2).

4. Overwrite the wall/freestream rows with the boundary conditions (§7 table), respecting the node ordering.
5. Solve the eigenvalue problem (temporal (28) or spatial (32)); filter for the discrete mode (§10); read c_i or $-\alpha_i$.
6. Sweep parameters; extract $\{c_i = 0\}$ for neutral curves, or integrate $-\alpha_i$ for N -factors.

Code map. base flow: `pymack/baseflow.py`; scales: `pymack/scales.py`; spectral operators: `pymack/spectral.py`; temporal assembly: `pymack/temporal_solver.py`; spatial assembly: `pymack/equations.py`; solvers / companion / Newton: `pymack/solver.py`; mode selection: `discrete_mode.py`; neutral-curve drivers and continuation: `verification/.../build*_grid.py`, `extract_contours.py`, `trace_continuation.py`.